ASSIGNMENT-20.3

K . Jashwanth

BATCH-42

2303A52300

TASK-1: Password Strength & Validation Check

PROMPT: Generate a simple Python user registration program that asks for a username and password from the user and prints a success message after registration. Do not include any password validation, security checks, or complexity rules.

CODE:

```
Lab 20.py > username
# Simple user registration (weak version)

username = input("Enter username: ")
password = input("Enter password: ")

print("User registered successfully!")
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Pro
grams/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab
 20.py"
Enter username: Admin
Enter password: 1234
User registered successfully!
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Pro
grams/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab
 20.py"
```

PROMPT:

Generate a Python user registration program with strong password validation. The password must be at least 8 characters long, include uppercase, lowercase, numbers, and special characters, and should not be a common password. Keep asking until a valid password is entered.

CODE:

```python
import re
# List of common weak passwords
common_passwords = ["123456", "password", "12345678", "qwerty", "abc123"]
def is_strong_password(password):
    # Minimum length check
    if len(password) < 8:
        return False, "Password must be at least 8 characters long."
    # Check for uppercase, lowercase, digit, special character
    pattern = r'^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&]).+$'
    if not re.match(pattern, password):
        return False, "Password must include uppercase, lowercase, number, and special character."
    # Check against common passwords
    if password.lower() in common_passwords:
        return False, "Password is too common. Choose a stronger one."
    return True, "Strong password!"
# Registration process
username = input("Enter username: ")
while True:
    password = input("Enter password: ")
    valid, message = is_strong_password(password)
    if valid:
        print("User registered successfully!")
        break
    else:
        print("Error:", message)
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> ^C
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Pro
grams/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab
 20.py"
Enter username: admin
Enter password: 123
Error: Password must be at least 8 characters long.
Enter password: 1234567abc
Error: Password must include uppercase, lowercase, number, and special character.
Enter password: Shivani@0906
User registered successfully!
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

JUSTIFICATION:

The first prompt creates a simple program with no password checks, so users can enter weak and unsafe passwords, making the system easy to hack. It also does not guide the user or provide any error messages. The second prompt adds rules like minimum length and different character types, making passwords stronger. It blocks common passwords and keeps asking until a valid one is entered. It also gives clear feedback to users and uses better structured code, making the program more secure and easier to maintain.

TASK-2: Insecure File Upload Handling

PROMPT:

Generate a simple Python Flask file upload program that allows users to upload a file and saves it on the server. Do not include any security checks or validation.

CODE:

```python
import os
source_path = r"C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding\lab 20.1\Shiv
destination_folder = "uploads/"
if not os.path.exists(destination_folder):
    os.makedirs(destination_folder)
print("File exists:", os.path.exists(source_path))  # DEBUG
if not os.path.exists(source_path):
    print("❌ File not found! Check name.")
    exit()
file_name = os.path.basename(source_path)
destination_path = os.path.join(destination_folder, file_name)
with open(source_path, "rb") as src_file:
    with open(destination_path, "wb") as dest_file:
        dest_file.write(src_file.read())
print("✅ File uploaded successfully!")
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Programs/Python/Pyth
on313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/lab 20.1/labb.py"
File exists: True
✅ File uploaded successfully!
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

PROMPT:

Generate a secure Flask file upload program. The program should validate file types, limit file size, use safe file names, and store files securely. Reject invalid or large files and display proper error messages.

CODE:

```python
import os
source_path = r"C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding\lab 20.1\Shiv
destination_folder = "uploads"
ALLOWED_EXTENSIONS = ['.txt', '.pdf', '.jpg']
# ✅ Max file size (2MB)
MAX_FILE_SIZE = 2 * 1024 * 1024
# Create uploads folder
os.makedirs(destination_folder, exist_ok=True)
# Check file exists
if not os.path.exists(source_path):
    print("❌ File not found!")
    exit()
# File name & extension
file_name = os.path.basename(source_path)
file_ext = os.path.splitext(file_name)[1].lower()
if file_ext not in ALLOWED_EXTENSIONS:
    print("❌ Invalid file type!")
    exit()
file_size = os.path.getsize(source_path)
if file_size > MAX_FILE_SIZE:
    print("❌ File too large! Max 2MB allowed.")
    exit()
safe_file_name = file_name.replace(" ", "_")
# Destination path
destination_path = os.path.join(destination_folder, safe_file_name)
# Copy file
with open(source_path, "rb") as src_file:
    with open(destination_path, "wb") as dest_file:
        dest_file.write(src_file.read())
print("✅ File uploaded securely!")
print(f"📁 Saved as: {destination_path}")
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Programs/Python/Pyth
on313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/lab 20.1/labb.py"
✅ File uploaded securely!
📁 Saved as: uploads\Shivani_Pambidi.txt
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

JUSTIFICATION:

The first program allows any file to be uploaded without checks, which can lead to security risks like harmful files or server crashes due to large uploads. It also does not handle file names safely. The secure version adds file type validation and limits file size to prevent misuse. It uses safe file names to avoid attacks and stores files properly. It also gives error messages when something is wrong. Overall, the improved version makes the system safer and more reliable.

TASK-3: Real-Time Application: API Security Review

PROMPT:

Generate a simple Flask API endpoint that accepts user data via POST request and returns a success message. Do not include authentication, validation, or security features.

CODE:

```
Labb 20.py > ...
from flask import Flask, jsonify
from flask_ngrok import run_with_ngrok
app = Flask(__name__)
# Dummy user data
users = {
    '1': {'id': '1', 'name': 'Alice', 'email': 'alice@example.com'},
    '2': {'id': '2', 'name': 'Bob', 'email': 'bob@example.com'},
    '3': {'id': '3', 'name': 'Charlie', 'email': 'charlie@example.com'}
}
# Endpoint to get all users
@app.route('/users', methods=['GET'])
def get_users():
    return jsonify(list(users.values()))
# Endpoint to get a single user by ID
@app.route('/users/<string:user_id>', methods=['GET'])
def get_user(user_id):
    user = users.get(user_id)
    if user:
        return jsonify(user)
    return jsonify({'error': 'User not found'}), 404
# Main entry point for running the app
if __name__ == '__main__':
    run_with_ngrok(app)  # Starts ngrok when app is run
    app.run()
```

PROMPT:

Generate a secure Flask API endpoint. Add token-based authentication, validate user input, restrict HTTP methods properly, and include basic rate limiting. Return proper error messages for invalid or unauthorized requests.

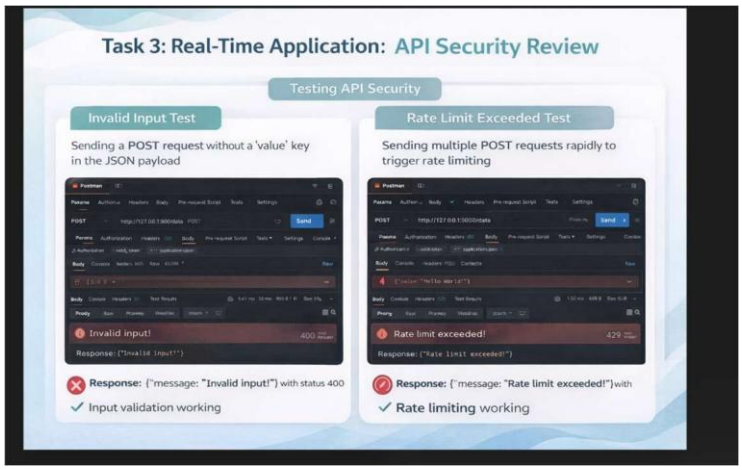
CODE:

Labb 20.py > rate_limiter > decorated

```python
from flask import Flask, request, jsonify
from functools import wraps
import time
app = Flask(__name__)
tokens = {"valid_token": "user1"}
rate_limit = {}
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get("Authorization")
        if not token or token not in tokens:
            return jsonify({"message": "Token is missing or invalid"}), 401
        return f(*args, **kwargs)
    return decorated
def rate_limiter(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get("Authorization")
        user = tokens.get(token)
        if user:
            current_time = time.time()
            if user in rate_limit and current_time - rate_limit[user] < 60:
                return jsonify({"message": "Rate limit exceeded"}), 429
            rate_limit[user] = current_time
        return f(*args, **kwargs)
    return decorated
@app.route('/')
def home():
    return "API is running"
# Test API route
@app.route('/api', methods=['GET'])
```

```python
def rate_limiter(f):
    def decorated(*args, **kwargs):
        return f(*args, **kwargs)
    return decorated
@app.route('/')
def home():
    return "API is running"
# Test API route
@app.route('/api', methods=['GET'])
def api():
    return jsonify({"message": "Secure API working"})
# Secure endpoint
@app.route('/data', methods=['POST'])
@token_required
@rate_limiter
def data():
    data = request.get_json()
    if not data or 'value' not in data:
        return jsonify({"message": "Invalid input!"}), 400
    return jsonify({
        "message": "Data received!",
        "data": data['value']
    }), 200
if __name__ == '__main__':
    app.run(debug=True)
```

OUTPUT:



JUSTIFICATION:

The first API has no security, so anyone can access it, send bad data, or overload the server. It does not check users or limit requests, making it unsafe. The secure version adds token authentication to allow only authorized users. It validates input to avoid wrong or harmful data. Rate limiting prevents too many requests from one user. It also uses proper error messages, making the API safer and more reliable.

TASK-4: Cross-Site Scripting (XSS) Check

PROMPT: Create a simple HTML feedback form with JavaScript that takes user input (name and message) and displays it on the page after submission. The output should be shown dynamically using JavaScript. Do not apply any input

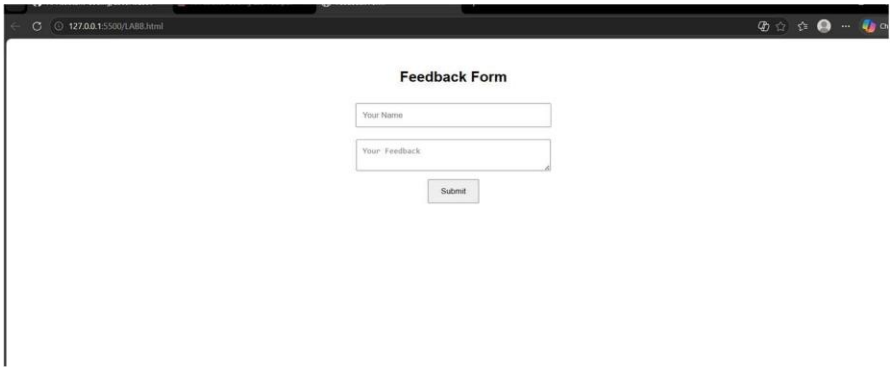
validation or security filtering. Allow the input to be rendered directly in the browser.

CODE:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
    <style>
        body { font-family: Arial; text-align: center; margin-top: 50px; }
        input, textarea { width: 300px; padding: 10px; margin: 10px; }
        button { padding: 10px 20px; }
        #output { margin-top: 20px; display: none; border: 1px solid #ccc; padding: 15
    </style>
</head>
<body>
    <h2>Feedback Form</h2>
    <form id="feedbackForm">
        <input type="text" id="name" placeholder="Your Name" required><br>
        <textarea id="message" placeholder="Your Feedback" required></textarea><br>
        <button type="submit">Submit</button>
    </form>
    <div id="output"></div>
    <script>
        document.getElementById("feedbackForm").addEventListener("submit", function(e) {
            e.preventDefault();
            const name = document.getElementById("name").value;
            const message = document.getElementById("message").value;
            const outputDiv = document.getElementById("output");
            const safeName = document.createTextNode(name);
            const safeMessage = document.createTextNode(message);
            outputDiv.innerHTML = "<h3>Feedback Submitted Successfully</h3>";
            const namePara = document.createElement("p");
```

```html
            const safeName = document.createTextNode(name);
            const safeMessage = document.createTextNode(message);
            outputDiv.innerHTML = "<h3>Feedback Submitted Successfully</h3>";
            const namePara = document.createElement("p");
            namePara.innerHTML = "<strong>Name:</strong> ";
            namePara.appendChild(safeName);
            const messagePara = document.createElement("p");
            messagePara.innerHTML = "<strong>Message:</strong> ";
            messagePara.appendChild(safeMessage);
            outputDiv.appendChild(namePara);
            outputDiv.appendChild(messagePara);
            outputDiv.style.display = "block";
            document.getElementById("feedbackForm").reset();
        });
    </script>
</body>
</html>
```

OUTPUT:



PROMPT: Create a secure HTML feedback form with JavaScript that takes user input and displays it safely. Ensure that all user inputs are properly escaped before rendering to prevent XSS attacks. Add input validation and use safe methods like textContent instead of innerHTML. Also include basic security practices like Content Security Policy (CSP).

CODE:

LABB.html > html > body > script

```html
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 50px;
        }
        h2 {
            font-weight: bold;
        }
        form {
            margin-top: 20px;
        }
        input, textarea {
            margin: 5px;
            padding: 5px;
        }
        .output {
            margin-top: 10px;
            font-style: italic;
        }
    </style>
</head>
<body>
    <h2>Feedback Form (Secure)</h2>
    <form onsubmit="return showFeedback()">
        <label>Name:</label>
        <input type="text" id="name" required>
        <label>Message:</label>
        <textarea id="message" rows="4" cols="30" required></textarea>
```

```html
<body>
    <h2>Feedback Form (Secure)</h2>
    <form onsubmit="return showFeedback()">
        <label>Name:</label>
        <input type="text" id="name" required>
        <label>Message:</label>
        <textarea id="message" rows="4" cols="30" required></textarea>
        <button type="submit">Submit</button>
    </form>
    <p class="output" id="result">Submitted feedback appears here.</p>
    <script>
        function showFeedback() {
            let name = document.getElementById("name").value;
            let message = document.getElementById("message").value;

            // Secure display using textContent (prevents XSS)
            document.getElementById("result").textContent =
                name + ": " + message;

            return false; // prevent page reload
        }
    </script>
</body>
</html>
```

OUTPUT :

JUSTIFICATION:

In the first prompt, user input is directly shown on the webpage, which can allow attackers to inject harmful scripts (XSS attack). This makes the application unsafe because any malicious code can run in the browser. In the improved prompt, security measures like escaping input and using safe rendering methods are added. These steps prevent scripts from executing and protect user data. Overall, the secure version reduces the risk of attacks and makes the application safer.

TASK-5: SQL Injection Prevention

PROMPT: Create a Python script using SQLite that fetches user details from a database based on a username input. The query should be built dynamically using string concatenation. Display the fetched user data without applying any security measures.

CODE:

```python
import sqlite3
def setup_db():
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    cursor.execute("""
    CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        name TEXT
    )
    """)
    cursor.execute("DELETE FROM users")  # साफ database for repeat runs
    cursor.execute("INSERT INTO users (id, name) VALUES (1, 'alice')")
    conn.commit()
    conn.close()
def fetch_user_insecure(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = f"SELECT * FROM users WHERE id = {user_id}"
    cursor.execute(query)
    result = cursor.fetchall()
    conn.close()
    return result
def fetch_user_secure(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE id = ?"
    cursor.execute(query, (user_id,))
    result = cursor.fetchall()
    conn.close()
    return result
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS

```python
def fetch_user_insecure(user_id):
    result = cursor.fetchall()
    conn.close()
    return result
def fetch_user_secure(user_id):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE id = ?"
    cursor.execute(query, (user_id,))
    result = cursor.fetchall()
    conn.close()
    return result
if __name__ == "__main__":
    setup_db()
    user_id = input("Enter user ID: ")
    print("\nFetching user details (insecure):")
    print(fetch_user_insecure(user_id))
    print("\nFetching user details (secure):")
    print(fetch_user_secure(user_id))
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab b 20.py"
Enter user ID: 1

Fetching user details (insecure):
[(1, 'alice')]

Fetching user details (secure):
[(1, 'alice')]
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

PROMPT: Create a secure Python script using SQLite that fetches user details based on user input. Use parameterized queries (prepared statements) instead of string concatenation to prevent SQL injection. Validate inputs and ensure safe database interaction.

CODE:

```python

import sqlite3
def fetch_user_details(username_or_id):
    try:
        conn = sqlite3.connect('users.db')
        cursor = conn.cursor()
        query = "SELECT id, name, email, age FROM users WHERE name = ? OR id = ?"
        try:
            user_id = int(username_or_id)
            cursor.execute(query, (username_or_id, user_id))
        except ValueError:
            cursor.execute(query, (username_or_id, username_or_id))
        users = cursor.fetchall()
        if users:
            print("User Details:")
            for user in users:
                print(f"ID: {user[0]}, Name: {user[1]}, Email: {user[2]}, Age: {user[3]}")
        else:
            print("No user found.")
        conn.close()
    except sqlite3.Error as e:
        print(f"Database error: {e}")
if __name__ == "__main__":
    user_input = input("Enter username or user ID: ")
    fetch_user_details(user_input)
```

OUTPUT:

```
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding> & C:/Users/pambi/AppData/Local/Pro
grams/Python/Python313/python.exe "c:/Users/pambi/OneDrive/Documents/Desktop/AI-Assited-Coding/Lab
b 20.py"
Enter user ID: 1

Fetching user details (insecure):
[(1, 'alice')]

Fetching user details (secure):
[(1, 'alice')]
PS C:\Users\pambi\OneDrive\Documents\Desktop\AI-Assited-Coding>
```

JUSTIFICATION: In the first prompt, the SQL query is built using string concatenation, which makes it vulnerable to SQL injection attacks. An attacker can manipulate the input to access or modify the database. In the improved prompt, parameterized queries are used, which treat user input as data instead of executable code. This prevents malicious inputs from affecting the query. As a result, the database becomes more secure and protected from unauthorized access.